

1. Relatório

- Alunos:
 - Daniel Wesleys Freitas Siqueira GRR20245621
 - Ulisses Bastian Machado da Rosa GRR20245567

2. Implementação

2.1. Kermit

Foi criada a seguinte estrutura de dados para representar um pacote do protocolo *Kermit*:

```
struct KermitPacket {
    struct {
        unsigned char init_marker = KERMIT_INIT_MARKER;
        unsigned char size : 5;
        unsigned char sequence : 6;
        PacketType type : 5; // enum com os tipos de mensagem
        // possíveis
    } header;
    char data[BUFFER_SIZE + 1] // esse vetor tem tamanho
    // suficiente para guardar uma mensagem de tamanho até 32B +
    // 1B
}
```

2.1.1. Construção do pacote

- Dados são escritos na seção *data*
- O CRC é calculado sobre o *header* inteiro e sobre *data* (até 32B) e é escrito após o último byte relacionado ao pacote, ou seja, se o tamanho indicado for 5, por exemplo, o CRC será escrito no sexto byte do vetor.
- O conteúdo binário do struct é copiado para um buffer maior, com os bytes *x81* e *x88* sendo escapados com *xff*.
- Pacotes enviados têm tamanho mínimo de `sizeof(header) + 1B` e tamanho máximo `sizeof(header) + 31B + 1B`.

Para envio de pacotes foram feitas duas funções:

```
// função que recebe um pacote e guarda na estrutura
KermitPacket;
// utiliza recv() da libc e, em caso de timeout, retorna
erro (que é utilizado mais à frente);
// também retorna outros erros que podem ser relevantes
(sequência errada, crc errado, etc.)
PacketError KermitPacket::receivePacket(int socket);
// função que envia os dados presentes na estrutura;
// utiliza send() da libc, e retorna erro caso haja erro
com essa função
PacketError KermitPacket::sendPacket(int socket);
```

2.2. Envio de mensagens

Para facilitar o processo de envio de mensagens, foi criada uma camada de abstração com 3 funções:

```
// recebe pacotes de uma mensagem e concatena o conteúdo
em um std::vector
PacketError KermitPacket::receive(int socket,
std::vector<char*> buffer);
// "quebra" os dados presentes em um buffer de bytes e
envia pacotes referentes a esses dados
// inicia a transmissão com uma mensagem de tipo
específico initialize
PacketError KermitPacket::send(int socket, PacketType
type, const char* data, unsigned int data_size);
// chamada sempre após send() para poder "trocar" o estado
do emissor e receptor
// responsável por checar se o outro lado recebeu o pacote
do tipo end_transmission
PacketError KermitPacket::confirmSend(int socket);
```

2.2.1. Timeout

A função *send()* também implementa um segundo timeout, pois podemos receber “lixo” dos raw sockets com *recv()*. Dessa forma, se ficarmos durante um 8s sem receber mensagens do protocolo *Kermit*, podemos considerar que o envio falhou e reenviamos o pacote atual.

2.2.2. Sincronização

Com intuito de sincronizar ambos os lados da transmissão, a função *confirmSend()* foi feita com o intuito de “trocar” o estado do emissor e receptor, ou seja, é assumido que quem recebe com *receive()* passará a

poder enviar após o emissor chamar *confirmSend()* e quem envia passará a ouvir depois que chamar *confirmSend()*.

2.2.3. Mensagens

- Mensagens podem ter tamanho variável.
- Cada mensagem é composta por pacotes:
 - Pacote inicial.
 - Número variável de pacotes.
 - Pacote final.
- Cada pacote segue o protocolo *Kermit*.

2.3. Pacman

- A visualização do PacMan é um quadrado em volta dele, ou seja, se o nível da visualização é 2, então o quadrado resultante terá dimensões 5 × 5.
- A IA dos fantasmas foi feita utilizando
 - Mão direita
 - Mão esquerda
 - Alternante (mão esquerda/direita)
 - Aleatório
- As posições das entidades dependem do que há no csv:
 - Se qualquer uma delas for definida já no csv, a posição se mantém.
 - As que não estiverem no csv são aleatoriamente determinadas após a leitura do grid.

2.4. Cliente e Servidor

- Servidor começa enviando
- Cliente começa
- O cliente, após receber uma mensagem com o nome do arquivo (tipo), cria um arquivo temporário *nome ++ _received.<tipo>*.
- Os arquivos recebidos são apagados assim que a visualização atual do arquivo é fechada.